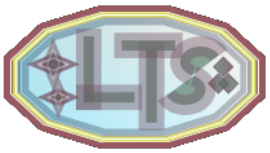


ConceptsDB-Imaging

A New Multimedia Database and Application Development Framework

ConceptsDB-Imaging (**ConceptsDB*i***) is a Multimedia Database and Application Development Framework designed by Linguistic Technology Systems (LTS). **ConceptsDB*i*** has an unconventional database design prioritizing storage of multimedia files (such as images, video, and **3D** graphics) and/or complex scientific/technical data structures. In addition to the underlying database engine, **ConceptsDB*i*** would provide application-development tools to facilitate programming software components that employ the database engine for persistence and query/search capabilities.

Multimedia databases have applications in a wide range of industrial and scientific areas. For example, image databases may be part of the software infrastructure used by engineers to track the progress of a construction project; certify the performance of industrial parts and products; document civic, manufacturing, or energy infrastructure; or build geospatial maps siting infrastructural data in the context of Geographic Information Systems (**GIS**).

ConceptsDB*i* provides a framework where multiple software applications can be networked together via multimedia and image-related extensions. The images handled by such applications can come from many sources. For example, images may be derived from videos, bioimaging, **3D** graphics, Panoramic Photography, and Computer Aided Design (**CAD**), as well as conventional photographs and microscopy. Projects using **ConceptsDB*i*** can extract images from multiple applications serving as import-sources, and load these images into a central application where images may be stored, tagged, annotated, and exposed to analytic engines such as **ITK** or **OpenCV**.

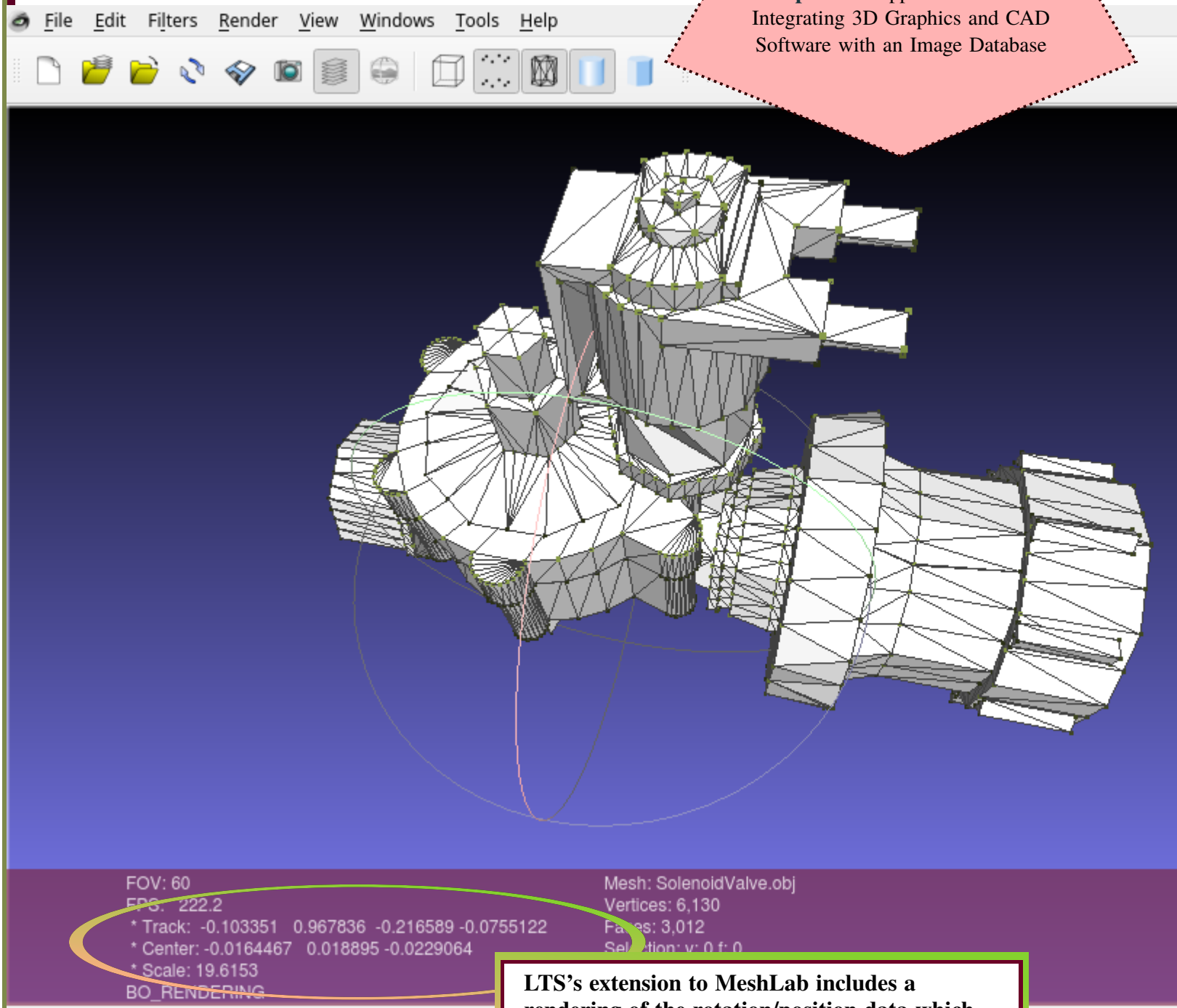
The screenshots depicted on the following slides are captured from **ConceptsDB*i*** components or extensions to pre-existing applications, demonstrating key features of application networks that could be implemented with **ConceptsDB*i***.

Use cases described in the slides below:

- Use Case 1: Synchronizing Image Databases with Industry Foundation Classes (IFC) and Building Information Management (BIM) Services
- Use Case 2: Image Metadata and Acquisition Context
- Use Case 3: IFC to Mesh Conversion for CAD
- Use Case 4: Computer Vision/AI Integration
- Use Case 5: WebGL Signal Processing
- Use Case 6: GIS Maps
- Use Case 7: OpenSCAD Integration
- Use Case 8: QCAD/Rendering Integration
- Use Case 9: QCAD/BRL-CAD Integration
- Use Case 10: VisIt Integration
- Use Case 11: Networking via the Forge API
- Use Case 12: Integration with Qt Signals/Slots
- Use Case 13: CaPTk Integration
- Use Case 14: Integration Controllers

ConceptsDB*i* **extensions or plugins would allow images to be extracted from existing applications, such as technologies used for Computer Aided Design or 3D graphics.**

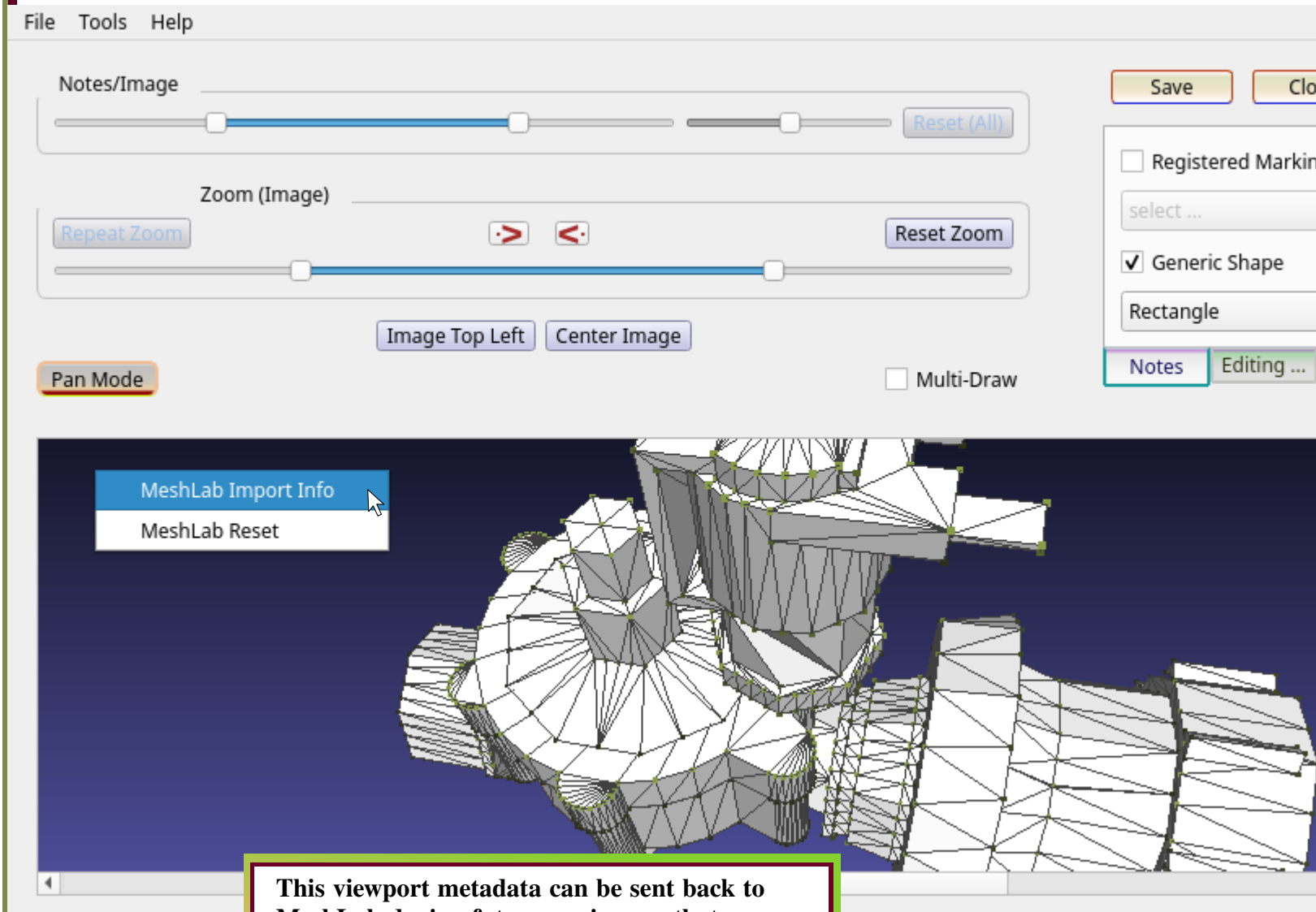
ConceptsDB*i* Application Networks: Integrating 3D Graphics and CAD Software with an Image Database



LTS's extension to MeshLab includes a rendering of the rotation/position data which governs how the MeshLab viewport constructs a 2D image from a 3D model.

In this example an image extracted from MeshLab would be automatically passed to a central image annotation and storage component. The imported data package would include viewport information which can be saved in the image database alongside the graphics file, documenting how the image was obtained from its 3D model source.

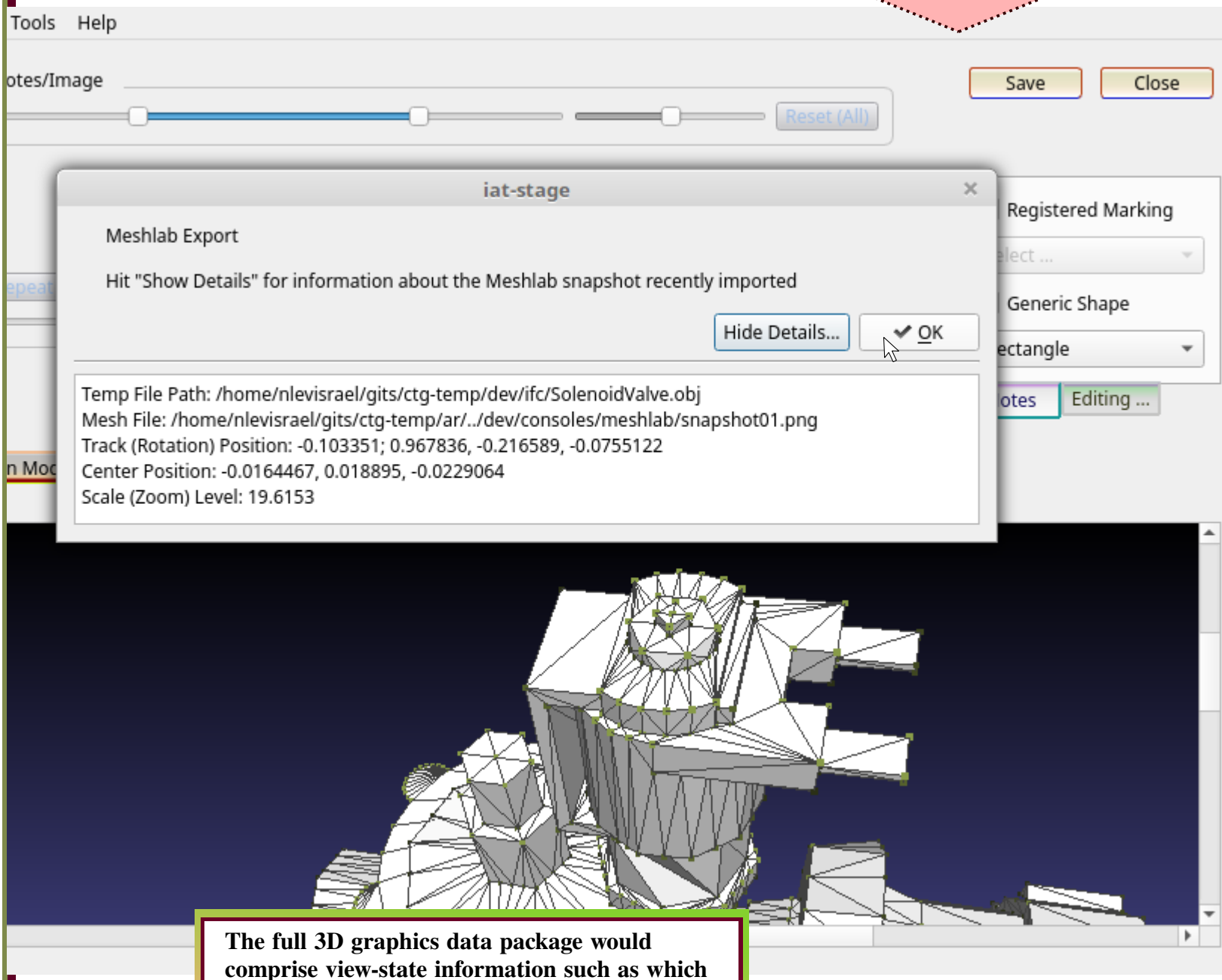
Use case 1: Synchronizing Image Databases with Industry Foundation Classes (IFC) and Building Information Management (BIM) Services



This viewport metadata can be sent back to MeshLab during future sessions so that users of the image database would be able to recreate in MeshLab the specific view-state from which the stored image had originated.

Using ConceptsDB*i*, MeshLab viewport information, which would stored alongside raw images (as described above) could then be previewed and examined while viewing the corresponding images or shared over a network to synchronize view-state among multiple users.

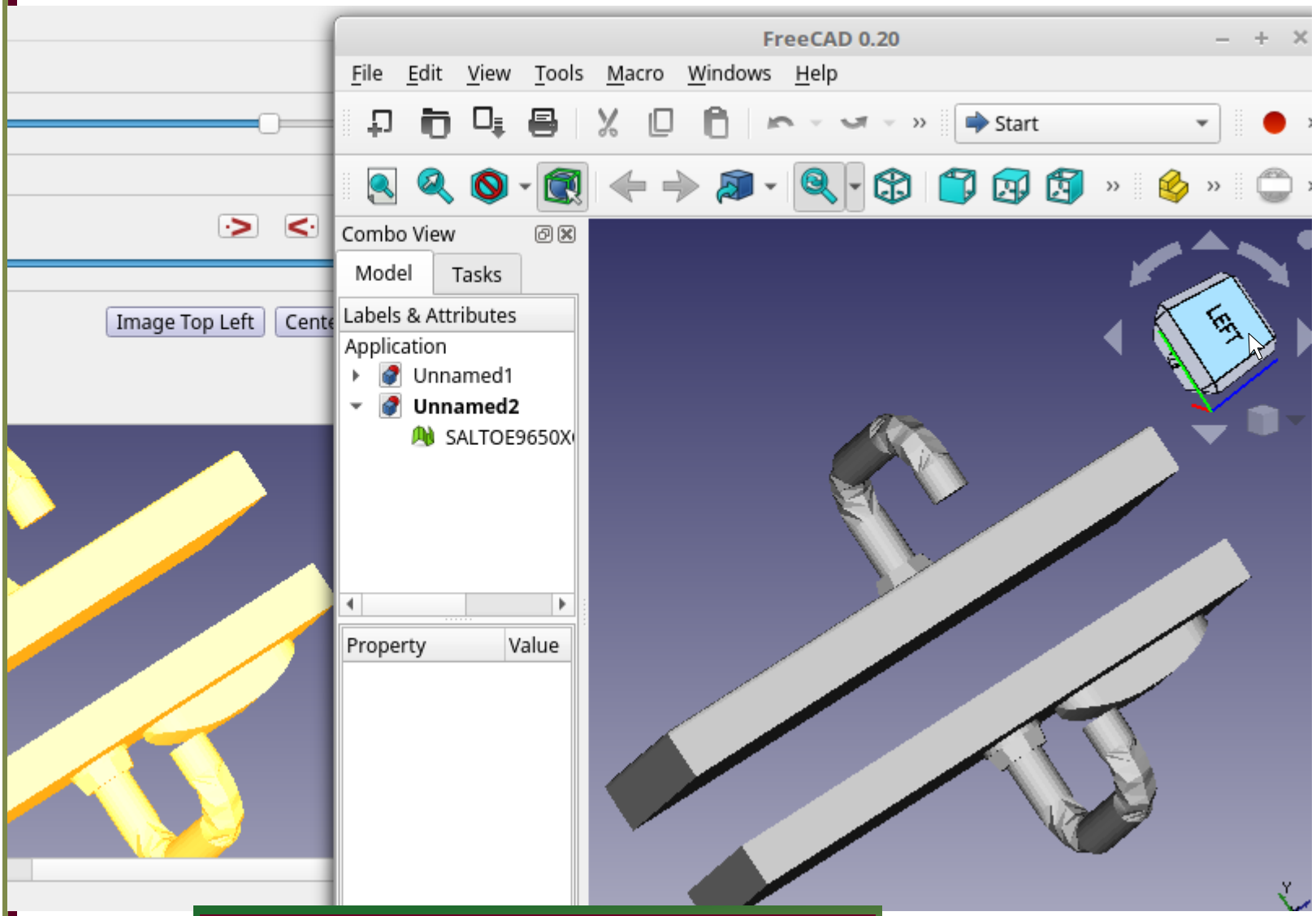
Use case 2: Image Metadata and Acquisition Context



The full 3D graphics data package would comprise view-state information such as which visual elements are activated within the image (e.g., mesh lines, point clouds, and lighting/ shading effects) as well as texture sources.

ConceptsDB*i* **images could be extracted from Computer Aided Design applications using an automated multi-application network, via ConceptsDB*i*** **plugins/extensions, with automated message-passing similar to data-sharing between an image database and 3D graphics software. This screenshot displays extensions to FreeCAD, a popular open-source CAD program.**

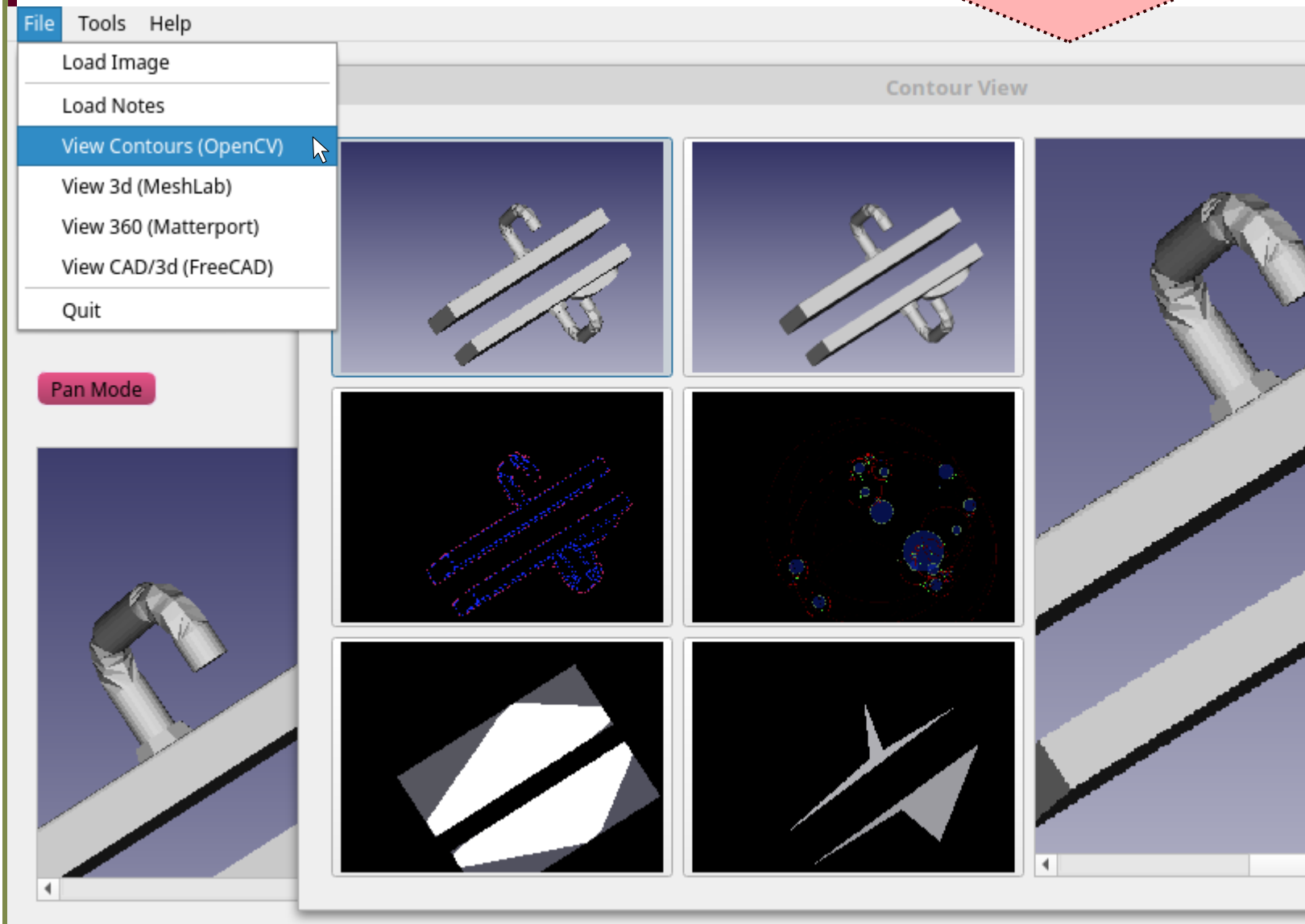
Use case 3: IFC to Mesh Conversion for CAD



In the case of CAD-derived images, metadata that would be stored in the image database (alongside raw image files) would contain construction/ materials details as well as viewport/visual state information.

Application Networks implemented via ConceptsDB*i* may interface with AI-driven image-processing libraries alongside standalone graphics/design software. In this example an image extracted from FreeCAD is sent to a component built via OpenCV, a computer-vision library, which renders several graphics depicting statistical data about contours and Image Regions obtained via image analysis.

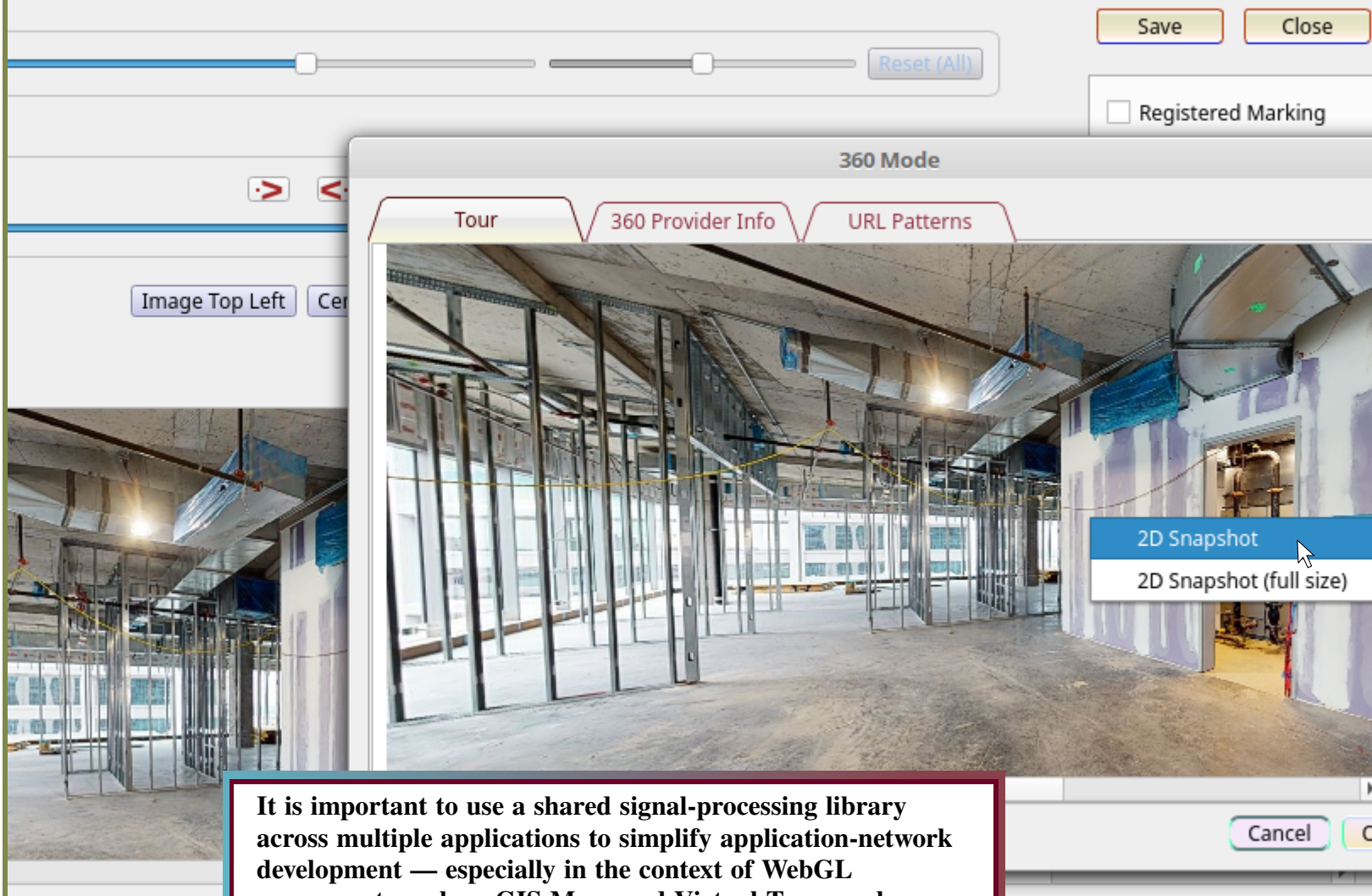
Use case 4: Computer Vision/AI Integration



In the context of an image database, this kind of Computer Vision statistical content could be mined for cross-corpus image similarity metrics, automated tagging, image-search capabilities, or automated Distinguished Region identification.

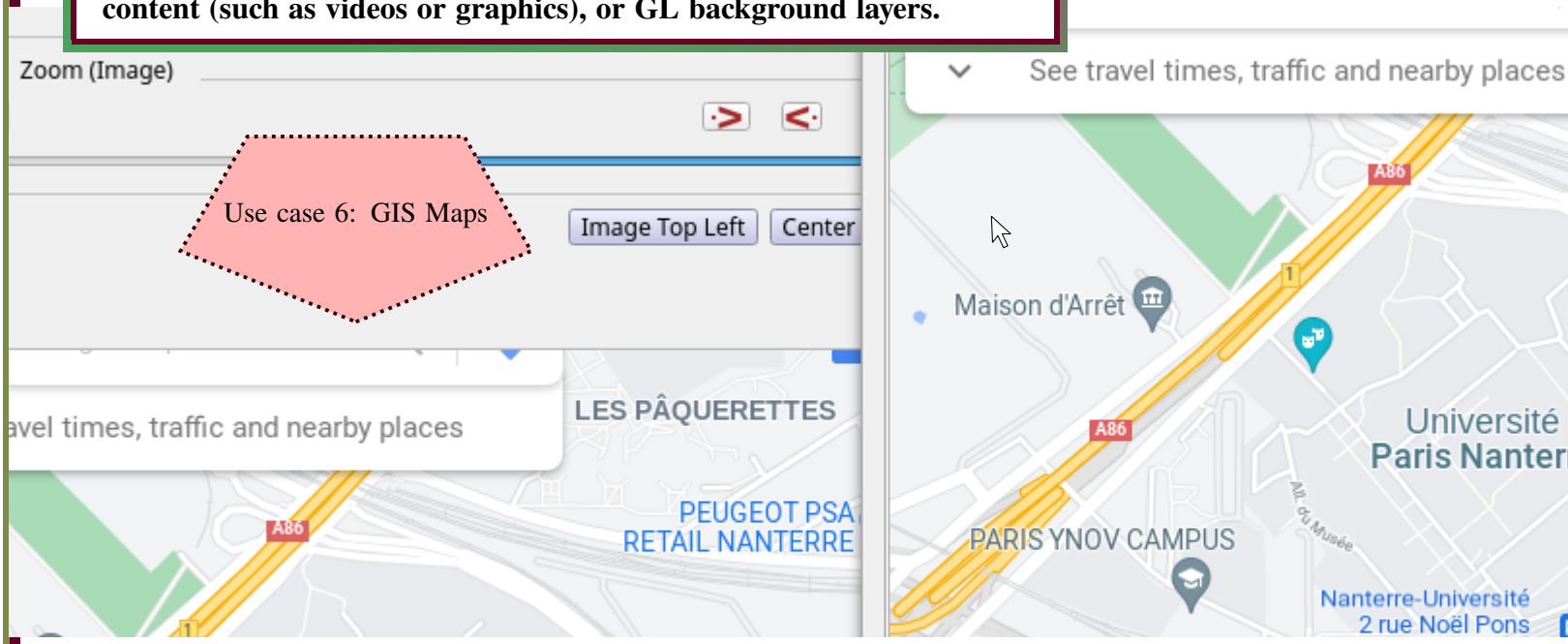
Use case 5: WebGL Signal Processing

This screenshot shows images being extracted from a Matterport Virtual Tour. ConceptsDB*i* would introduce a new signal-processing library to provide a consistent cross-application framework for handling user actions. This library may in turn be leveraged so as to provide users a consistent interface for extracting images and context meta-data from multiple applications.

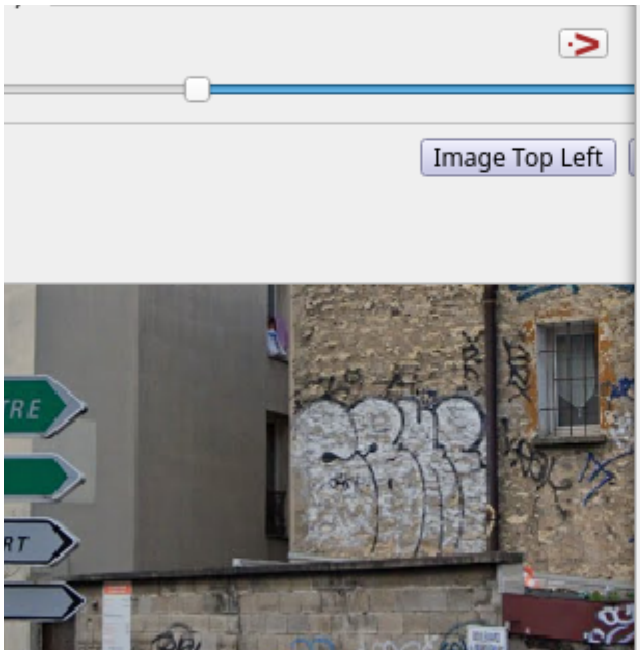


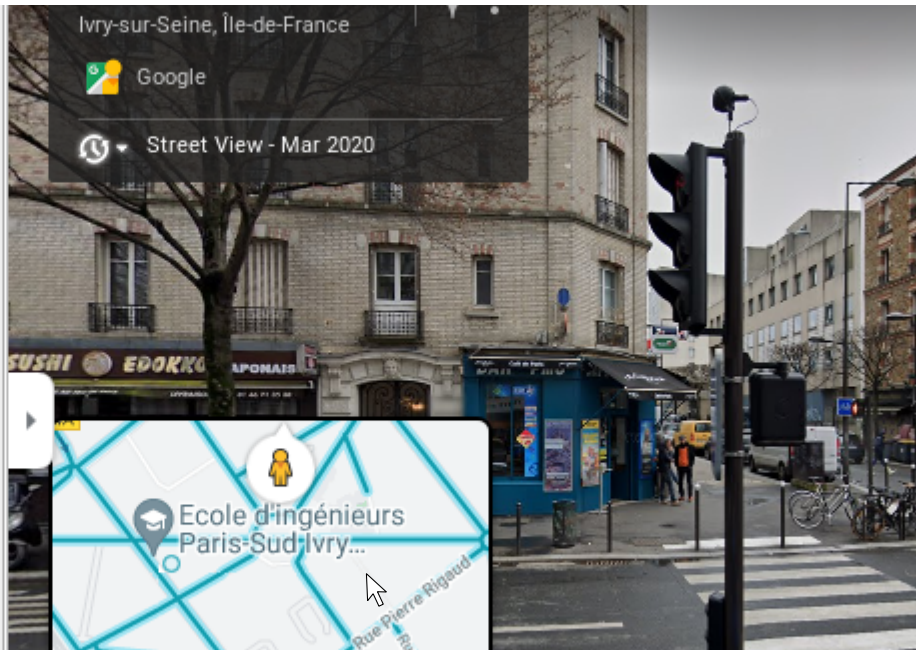
It is important to use a shared signal-processing library across multiple applications to simplify application-network development — especially in the context of WebGL components such as GIS Maps and Virtual Tours — because these components generate a complex mix of different user-action signals, which can lead to error-prone code duplication in a multi-application context. The best strategy for mitigating signal-handling errors is to consolidate signal-handling code into a single shared library.

Panoramic Photography, GIS, and Street Maps signals use different data structures and protocols depending on whether the mouse-event position corresponds with text inserts/tags, html links, multimedia content (such as videos or graphics), or GL background layers.



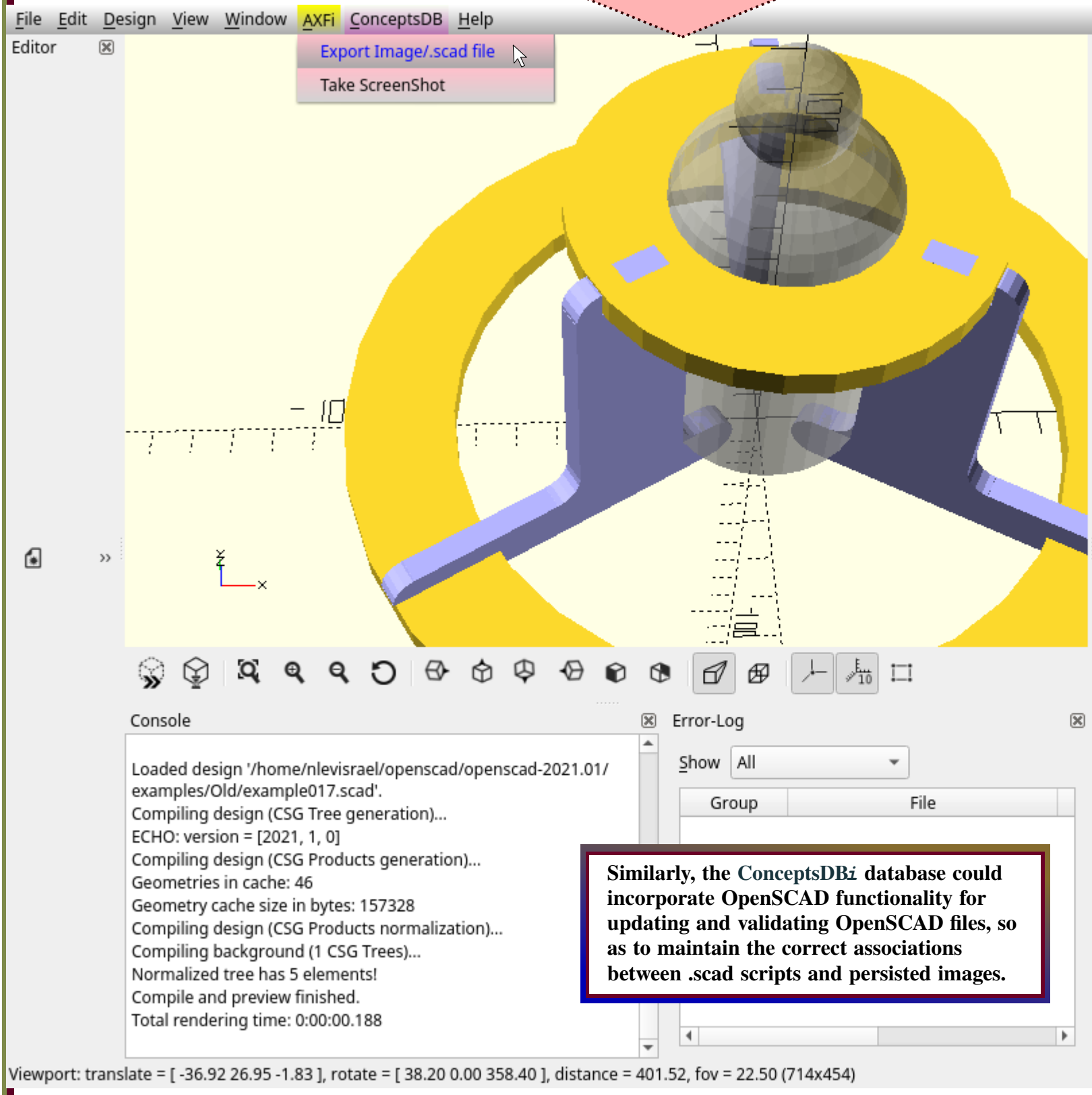
ConceptsDBi employs a custom runtime-reflection protocol to implement a convenient signal-handling layer that hides the complexities of different signal profiles across different points on an application network.





Using ConceptsDB*i*, extensions for OpenSCAD would allow images extracted from .scad renderings to be stored in a database alongside their corresponding OpenSCAD generator files.

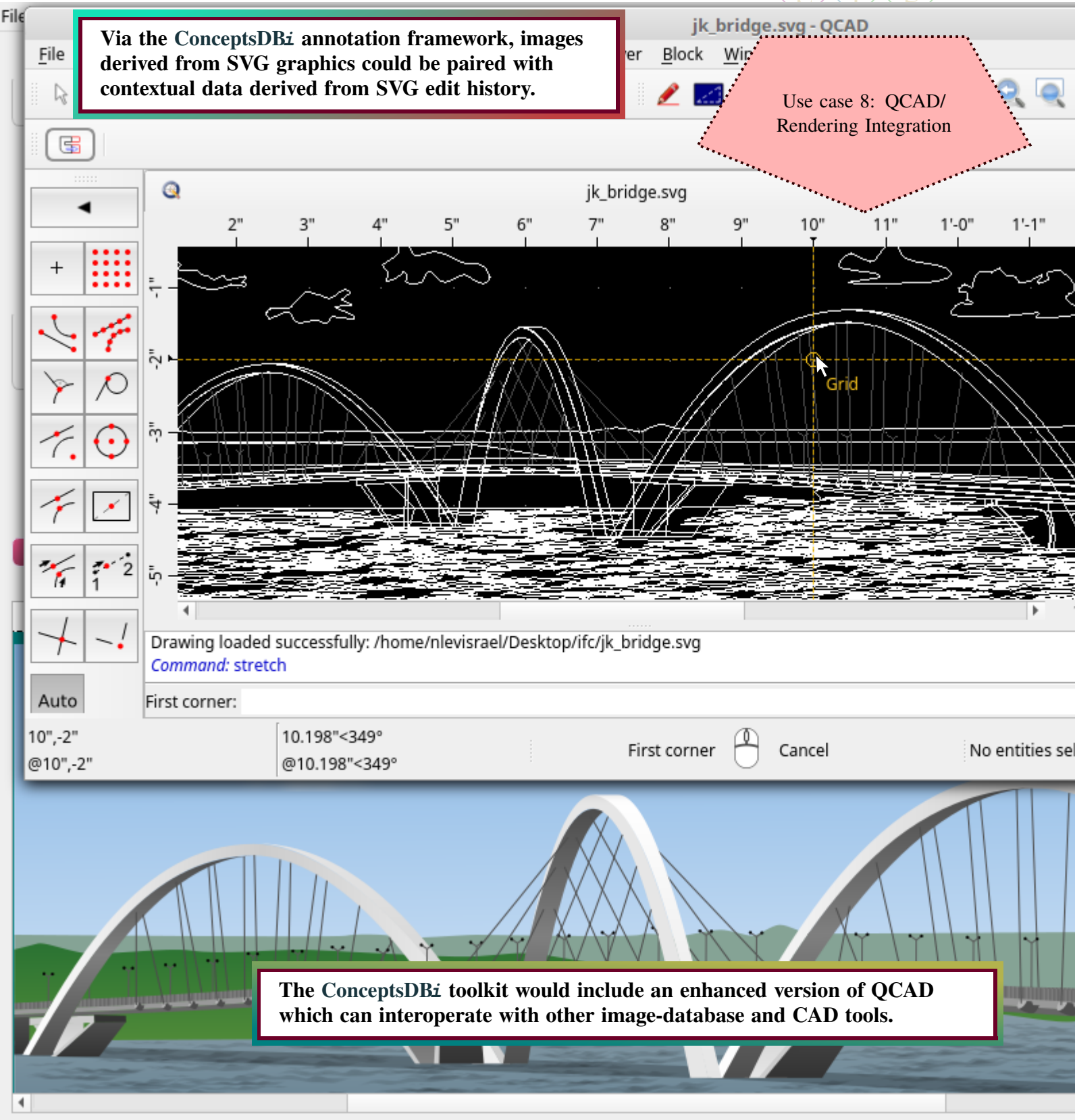
Use case 7:
OpenSCAD Integration



Similarly, the ConceptsDB*i* database could incorporate OpenSCAD functionality for updating and validating OpenSCAD files, so as to maintain the correct associations between .scad scripts and persisted images.

Via the ConceptsDB*i* annotation framework, images derived from SVG graphics could be paired with contextual data derived from SVG edit history.

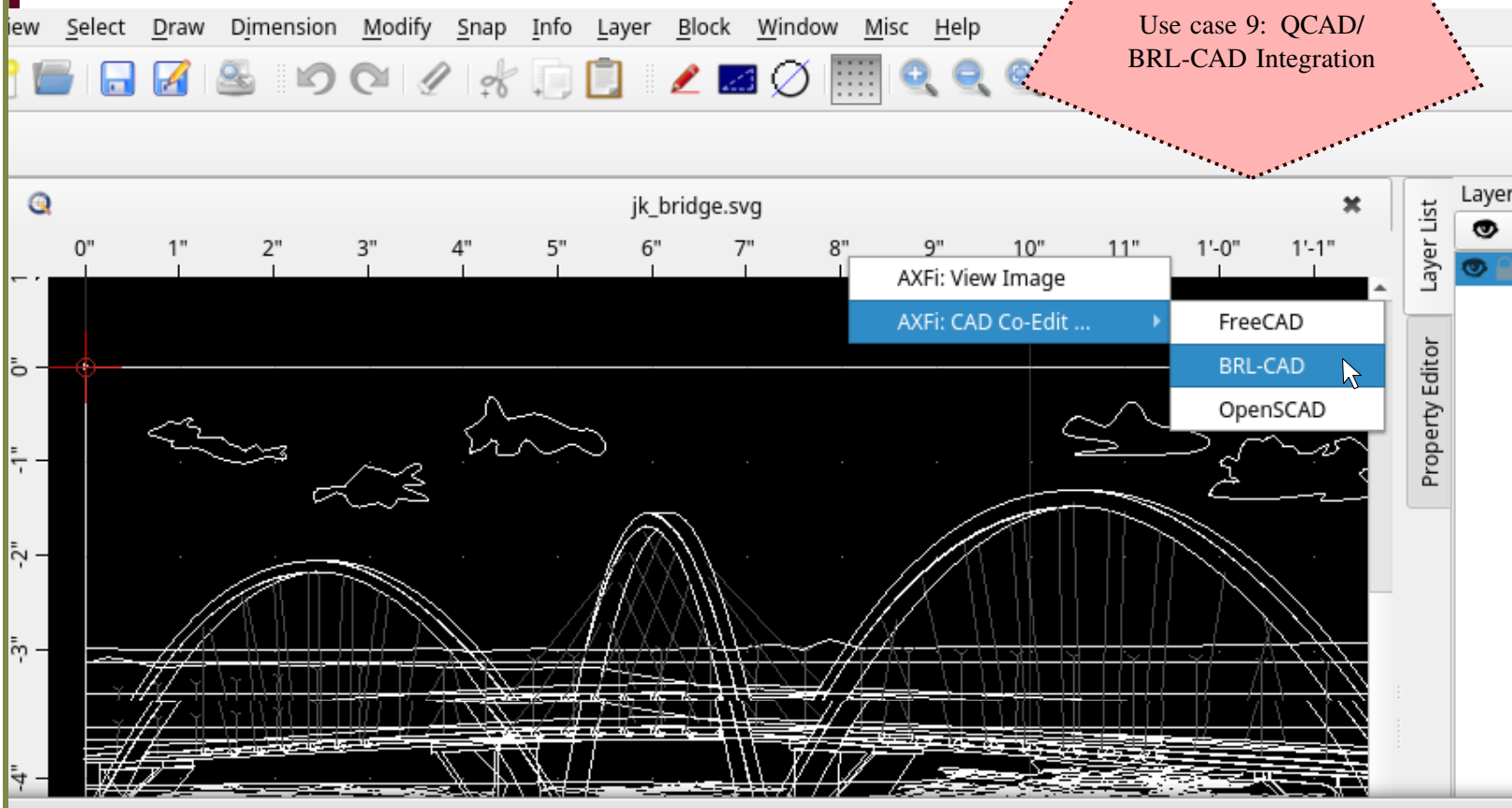
Use case 8: QCAD/ Rendering Integration

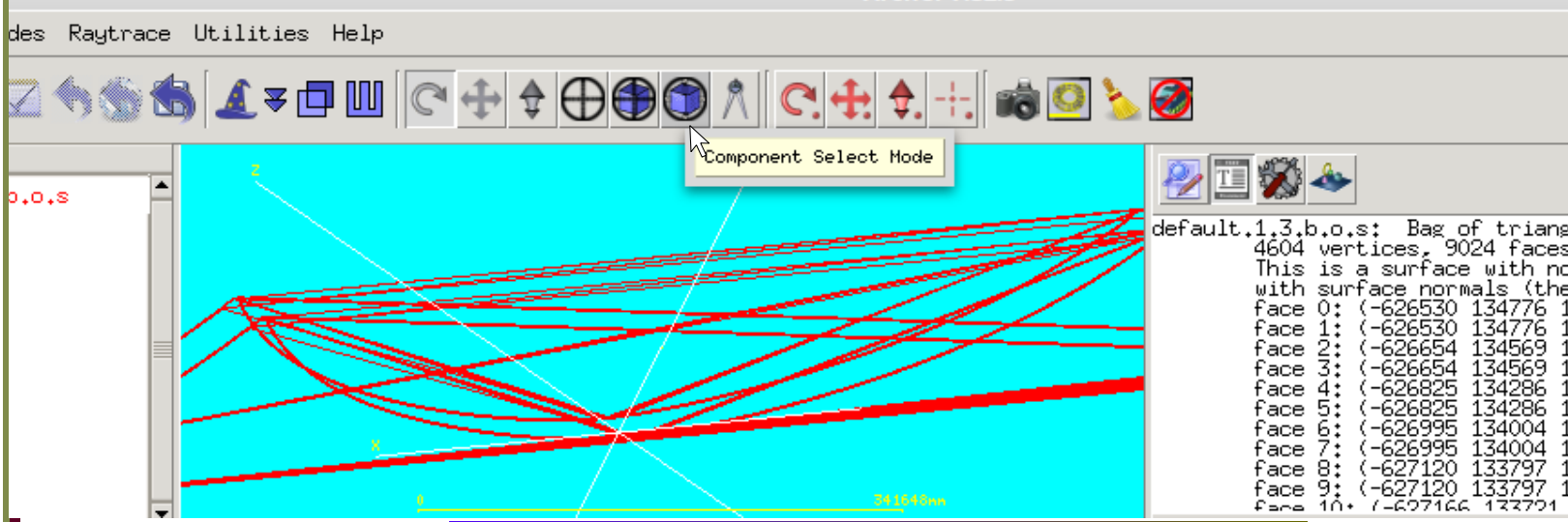


The ConceptsDB*i* toolkit would include an enhanced version of QCAD which can interoperate with other image-database and CAD tools.

ConceptsDB*i* **extensions to QCAD and BRL-CAD would allow these two applications to be used side-by-side for jointly editing CAD projects with a mixture of 2D and 3D graphics.**

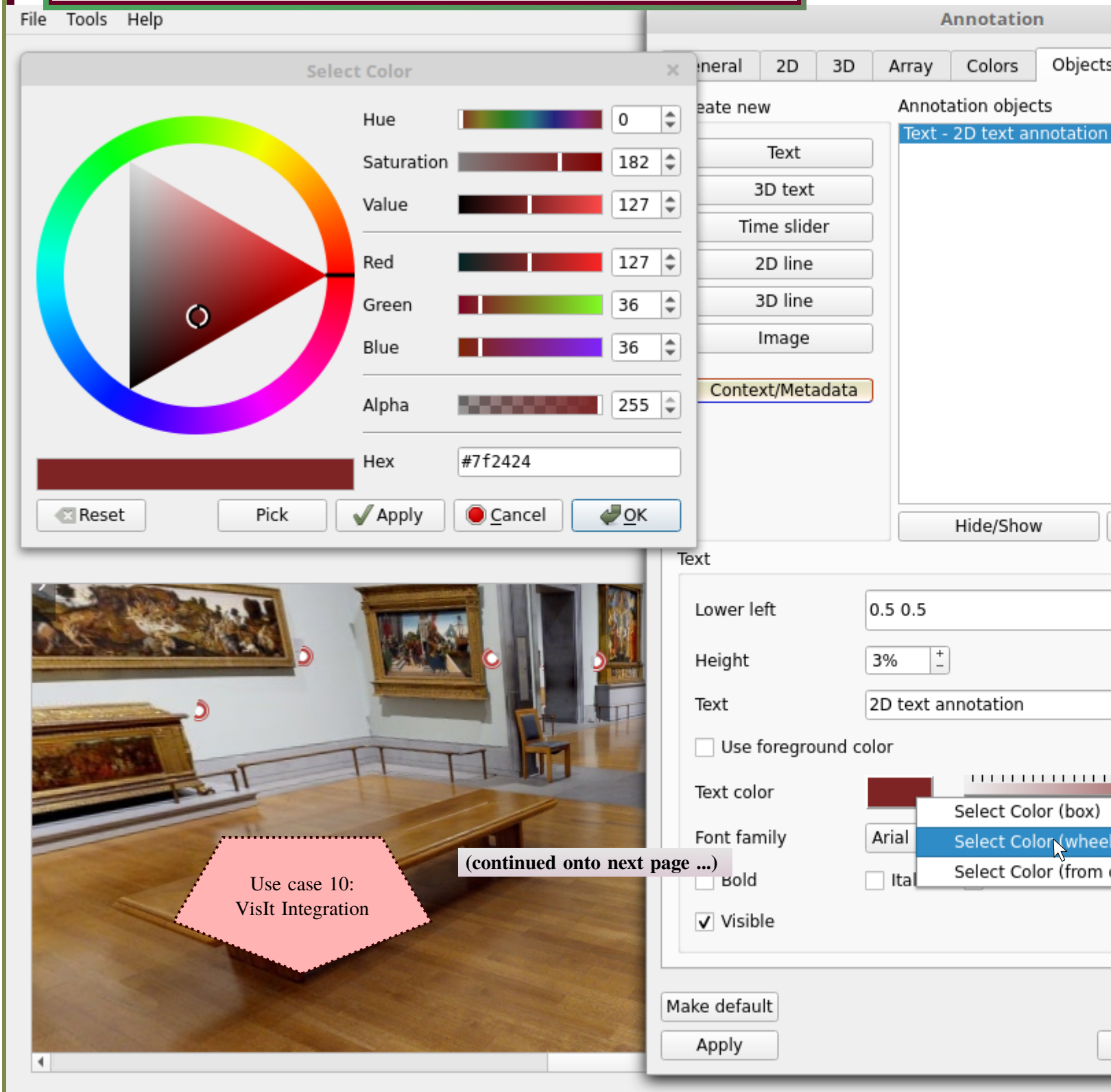
Use case 9: QCAD/ BRL-CAD Integration

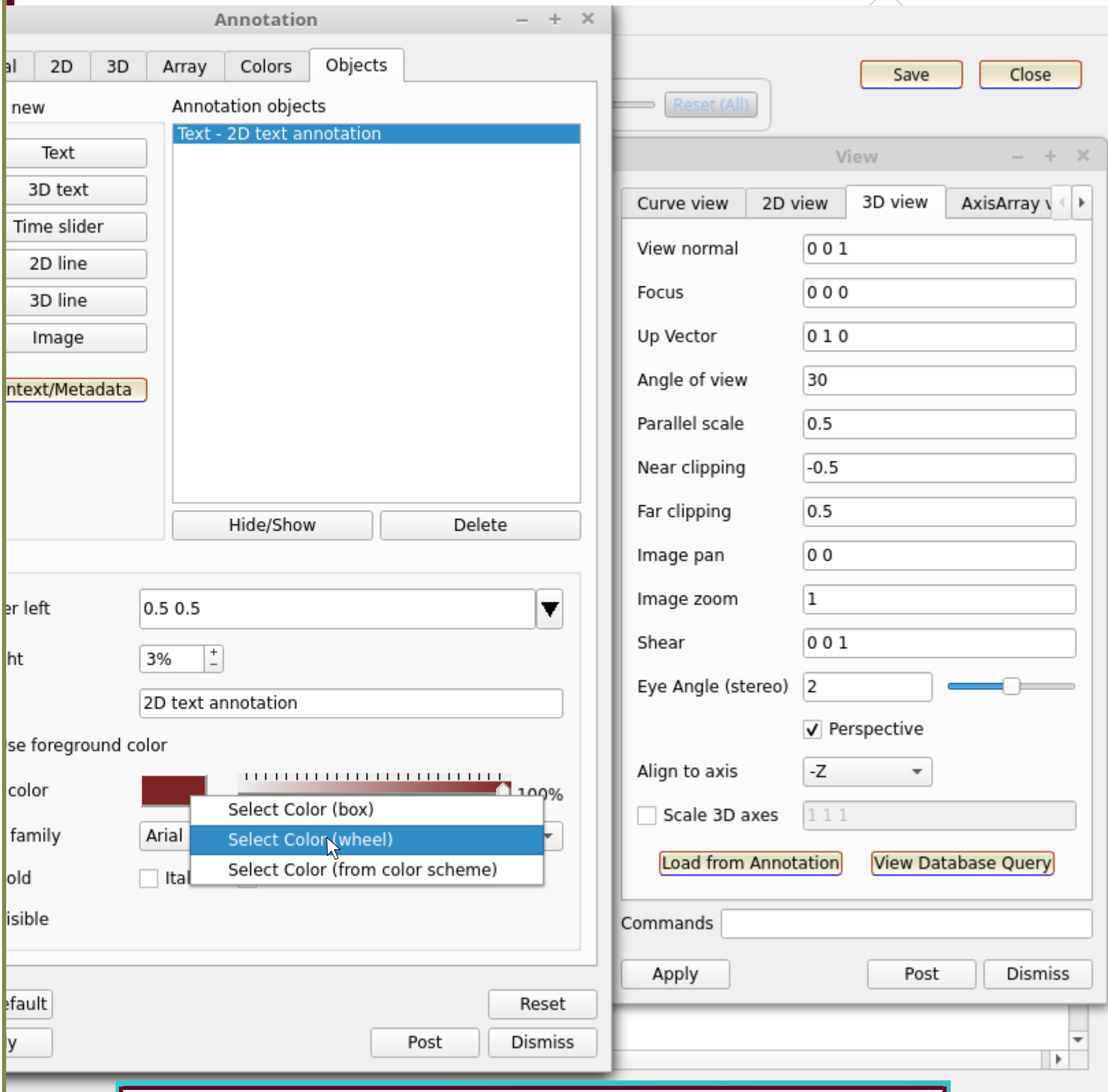




A ConceptsDB*i* **database could then save edit history merging both applications.**

To maintain consistent User Experience across multiple applications, ConceptsDB*i* would provide a suite of semi-autonomous GUI components based on existing CAD and data-visualization applications.

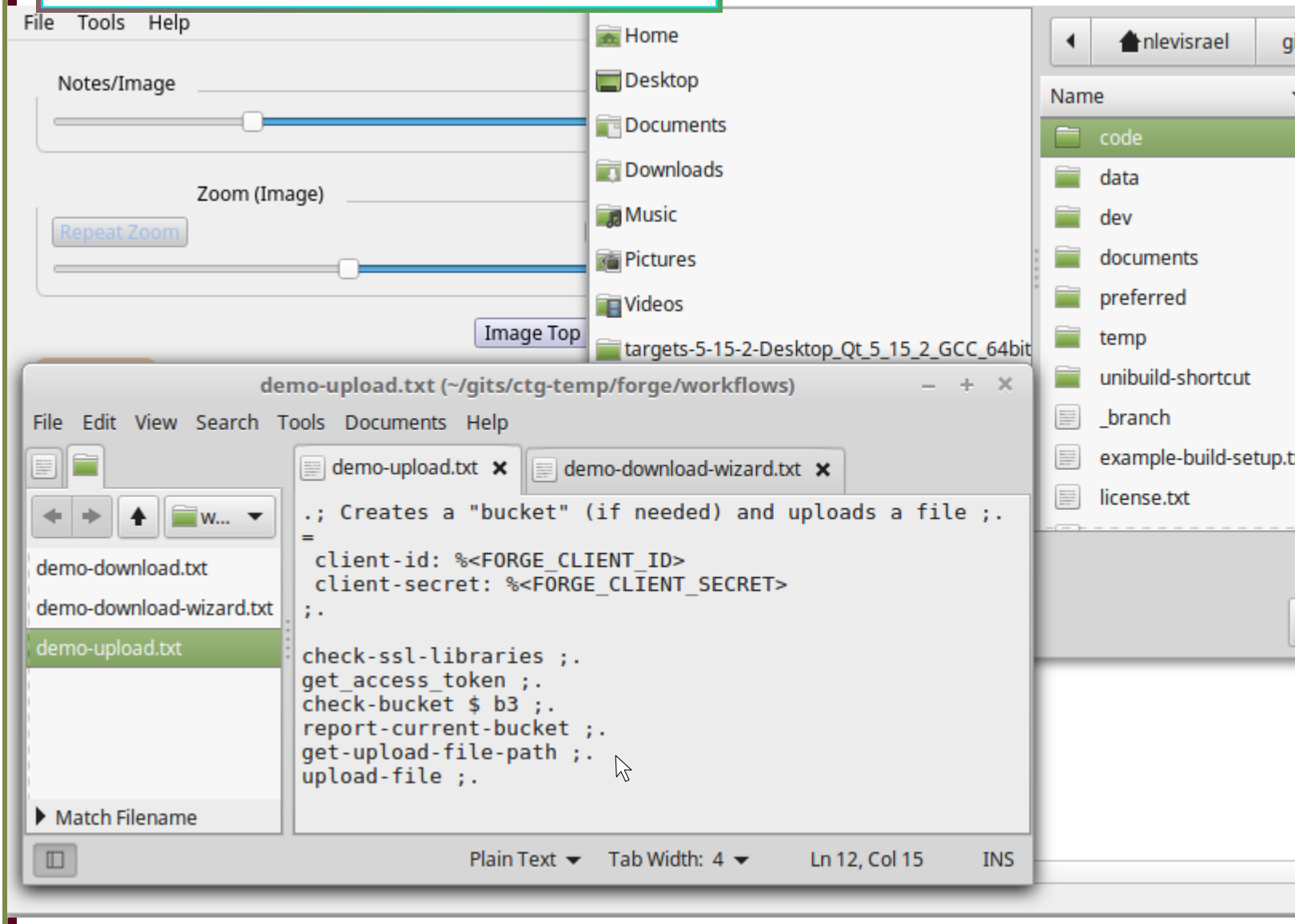




Using ConceptsDB*i*, tools such as annotation editors and viewport control consoles would be adopted from projects such as VisIt (the US Department of Energy Data-Visualization software) and modified to work in a multi-application context.

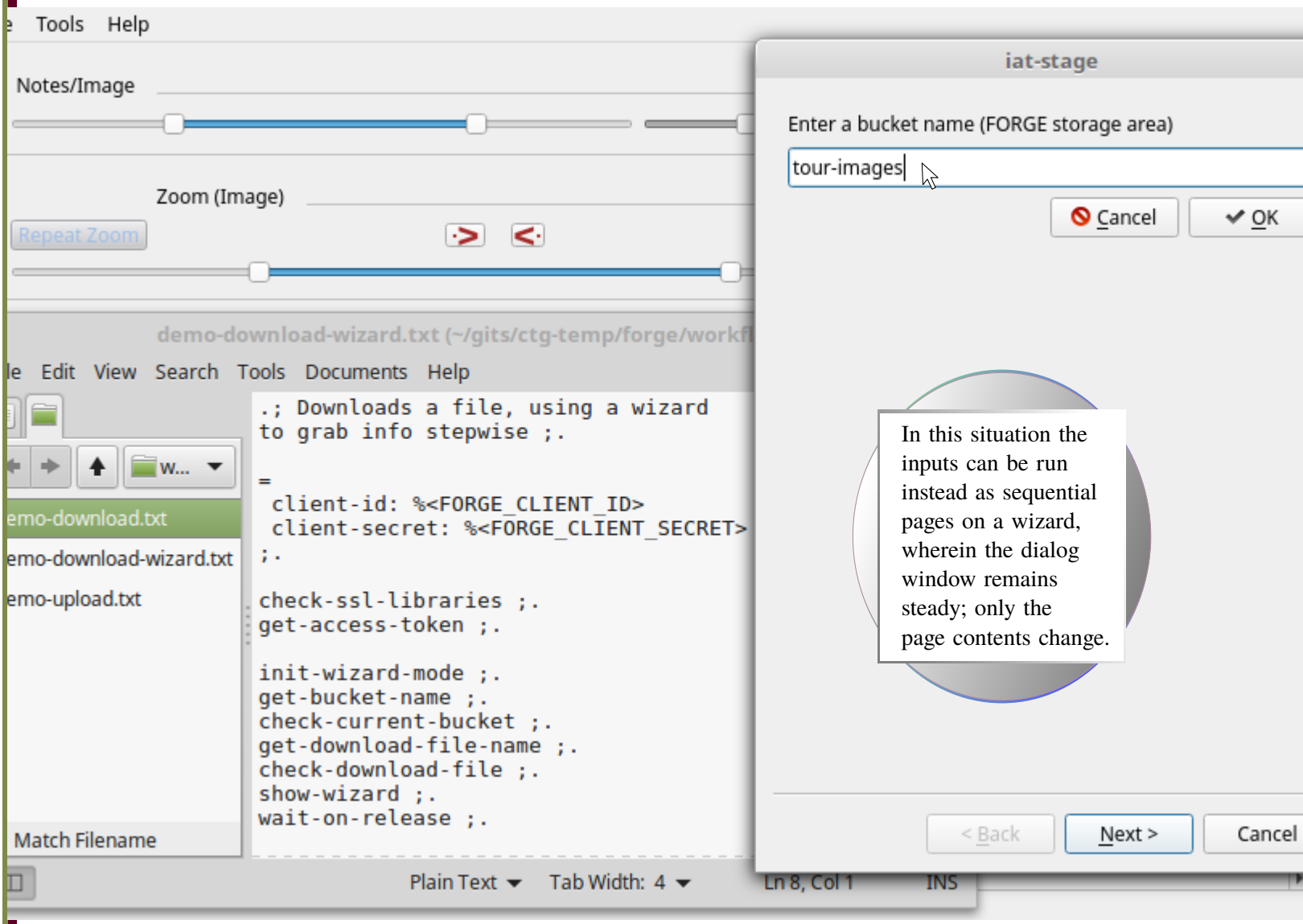
Additional **ConceptsDB*i*** features are designed to accommodate networking requirements such as interfacing with Autodesk Cloud applications via the Forge API. Because of how this API is structured, multiple operations often must be executed in a fixed sequence (for example, obtaining an access token prior to read/write operations), so that workflows cannot simply execute API calls via direct function calls. Instead, a callback mechanism must be used to ensure that each step in a workflow is only attempted after the prior step has completed.

Use case 11: Networking via the Forge API



To enable this form of staggered network protocol, the **ConceptsDB*i*** query-evaluation Virtual Machine can be extended with a workflow mechanism which manages the reactive-programming logic necessary to synchronize API calls behind the scenes.

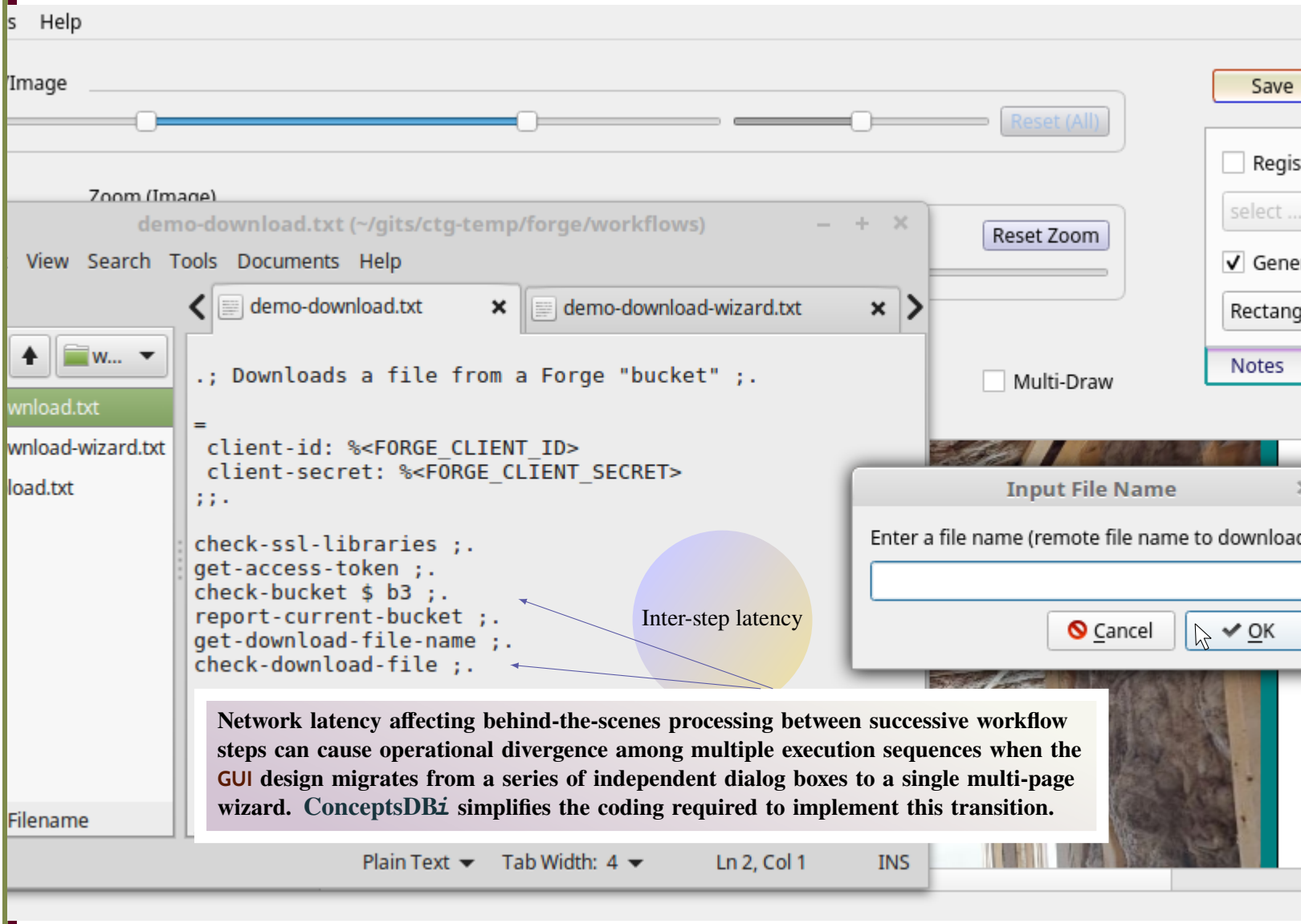
Virtual Machine extensions for network protocols such as the Forge API may also be integrated into a "wizard" mode, intended for exposing VM-backed workflows with wizard dialogs as part of a GUI. Portions of a workflow can then be flagged as communicating with the user via this wizard mode rather than exposing each step through a separate GUI window. This functionality would address use-cases where a workflow requires multiple user inputs, because opening and closing multiple input dialog boxes in succession can be disorienting for the end-user.



In this situation the inputs can be run instead as sequential pages on a wizard, wherein the dialog window remains steady; only the page contents change.

Wizards require special VM designs because each step in a wizard-backed workflow is situated within at least three different execution sequences; accordingly, callbacks need to be precisely orchestrated to ensure that data and/or files are exposed in the proper order.

Depending on developer preferences, Forge API workflows can acquire end-user data either via a sequence of independent GUI components or with successive Wizard pages.

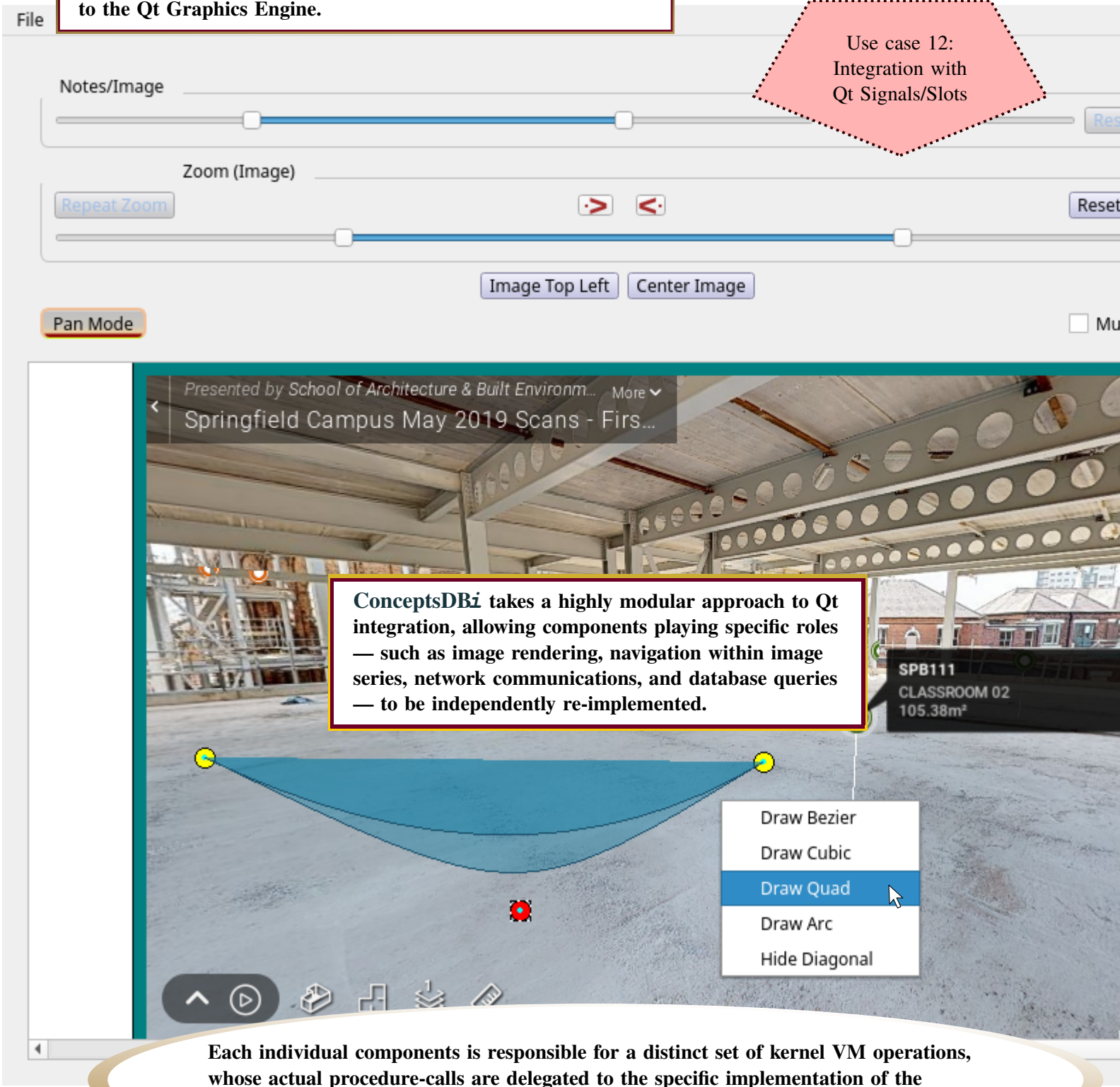


Network latency affecting behind-the-scenes processing between successive workflow steps can cause operational divergence among multiple execution sequences when the GUI design migrates from a series of independent dialog boxes to a single multi-page wizard. ConceptsDB*i* simplifies the coding required to implement this transition.

The workflow engine is designed to shield the implementation differences between these two distinct interactive modes from programmers writing code for **ConceptsDB*i*** applications, even though "wizard" mode requires extra layers of processing to synchronize divergent callback-sequences.

The ConceptsDBi Virtual Machine Query Engine introduces numerous extensions to Qt's reactive-programming framework to facilitate bridge code between Qt front-ends and networked resources (such as the Forge API) as well as query-based access to the Qt Graphics Engine.

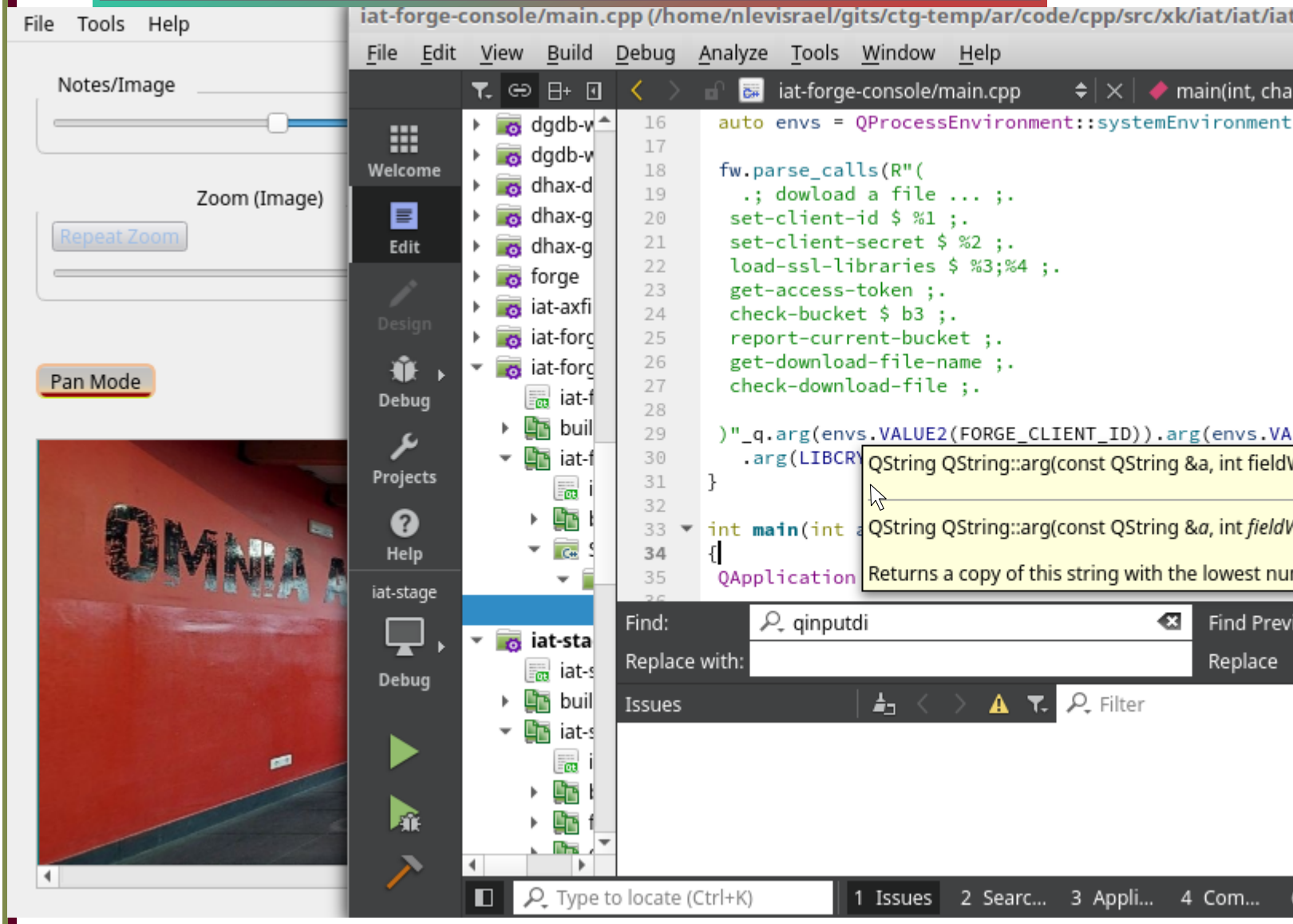
Use case 12:
Integration with
Qt Signals/Slots



ConceptsDBi takes a highly modular approach to Qt integration, allowing components playing specific roles — such as image rendering, navigation within image series, network communications, and database queries — to be independently re-implemented.

Each individual components is responsible for a distinct set of kernel VM operations, whose actual procedure-calls are delegated to the specific implementation of the relevant component which is adopted for a given ConceptsDBi project.

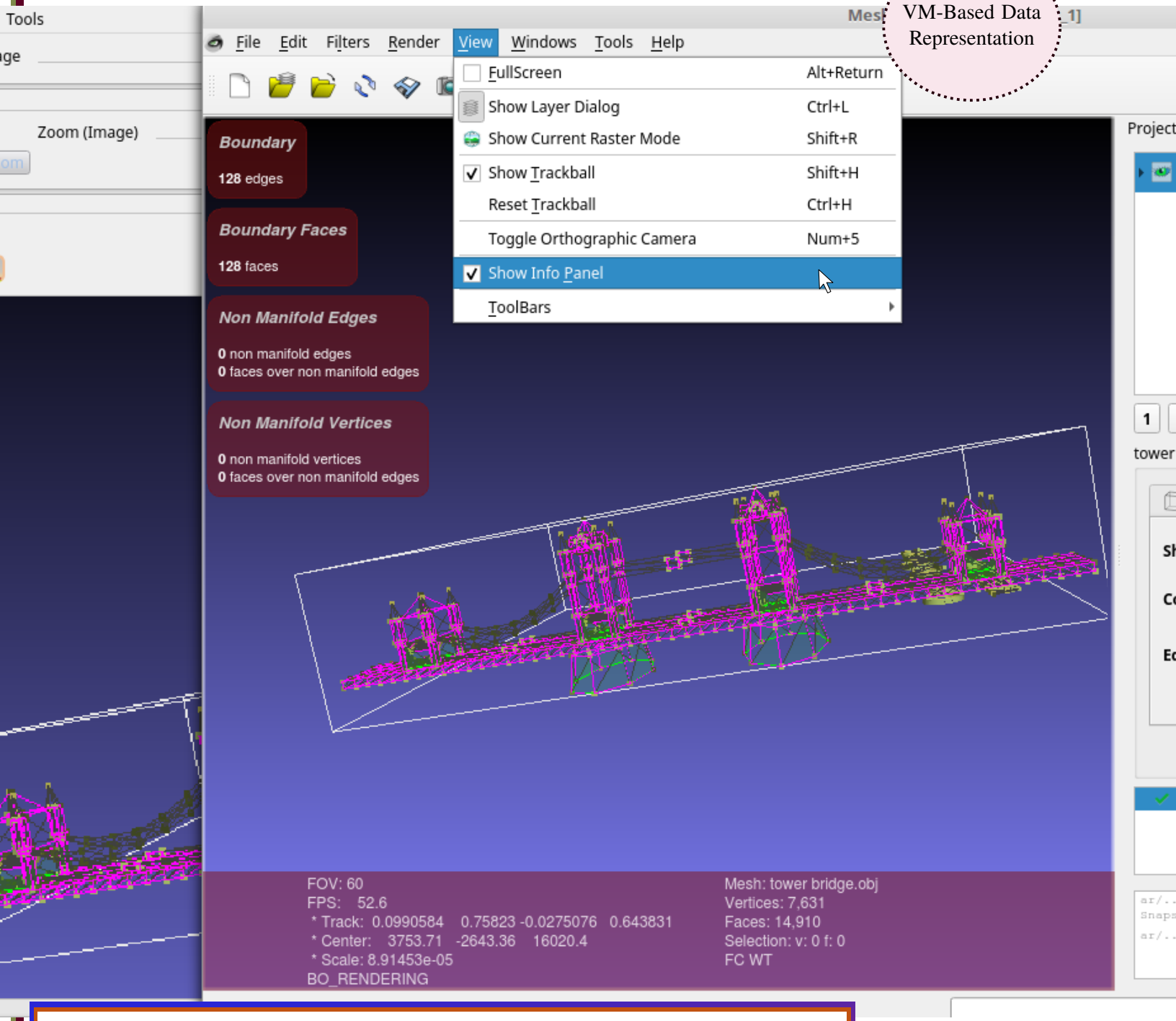
An additional benefit of the **ConceptsDBi** Virtual Machine/Networking system is that workflows can be described dynamically (e.g., via text files) even if the functions called by the workflows are statically-compiled C++ methods. The option of defining workflows directly in C++ code allows developers to fine-tune project-specific functions that can be exposed as kernel operations within VM extensions.



LTS can deliver innovative Hypergraph-based parsers and scripting engines to equip **ConceptsDBi** projects with custom query languages. Developers can thereby implement query-processing extensions as C++ procedures and then customize the syntax through which such functions would be expressed within query formats (which could be either inline DSL code or independent query languages).

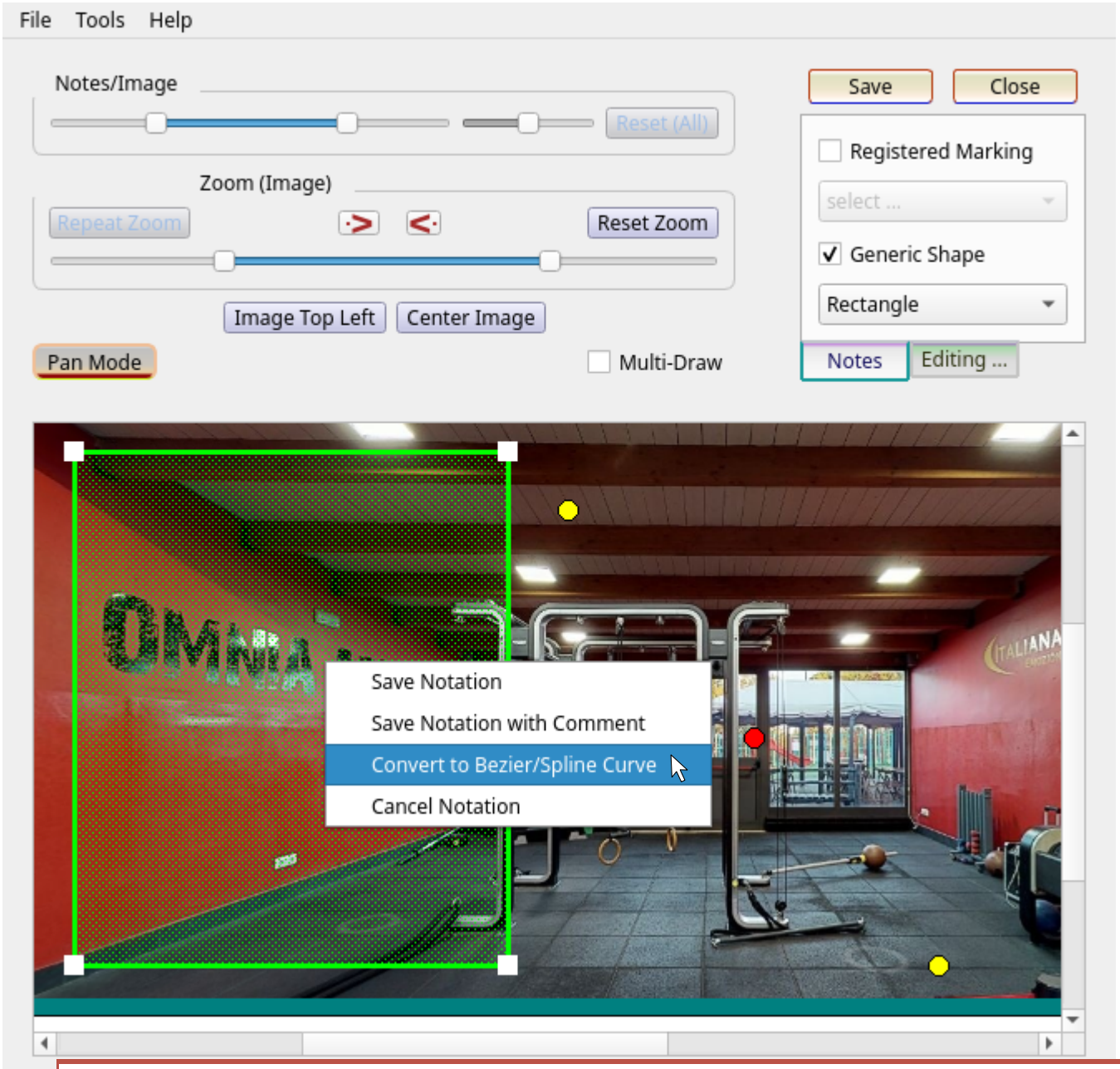
ConceptsDB*i* provides a VM-based parsing environment (in the sense that the steps needed to unpack serialized data structures into in-memory objects can be destructured into minimal operations suitable for VM operations) for reading data to be shared between components on a **ConceptsDB*i*** application network.

VM-Based Data Representation



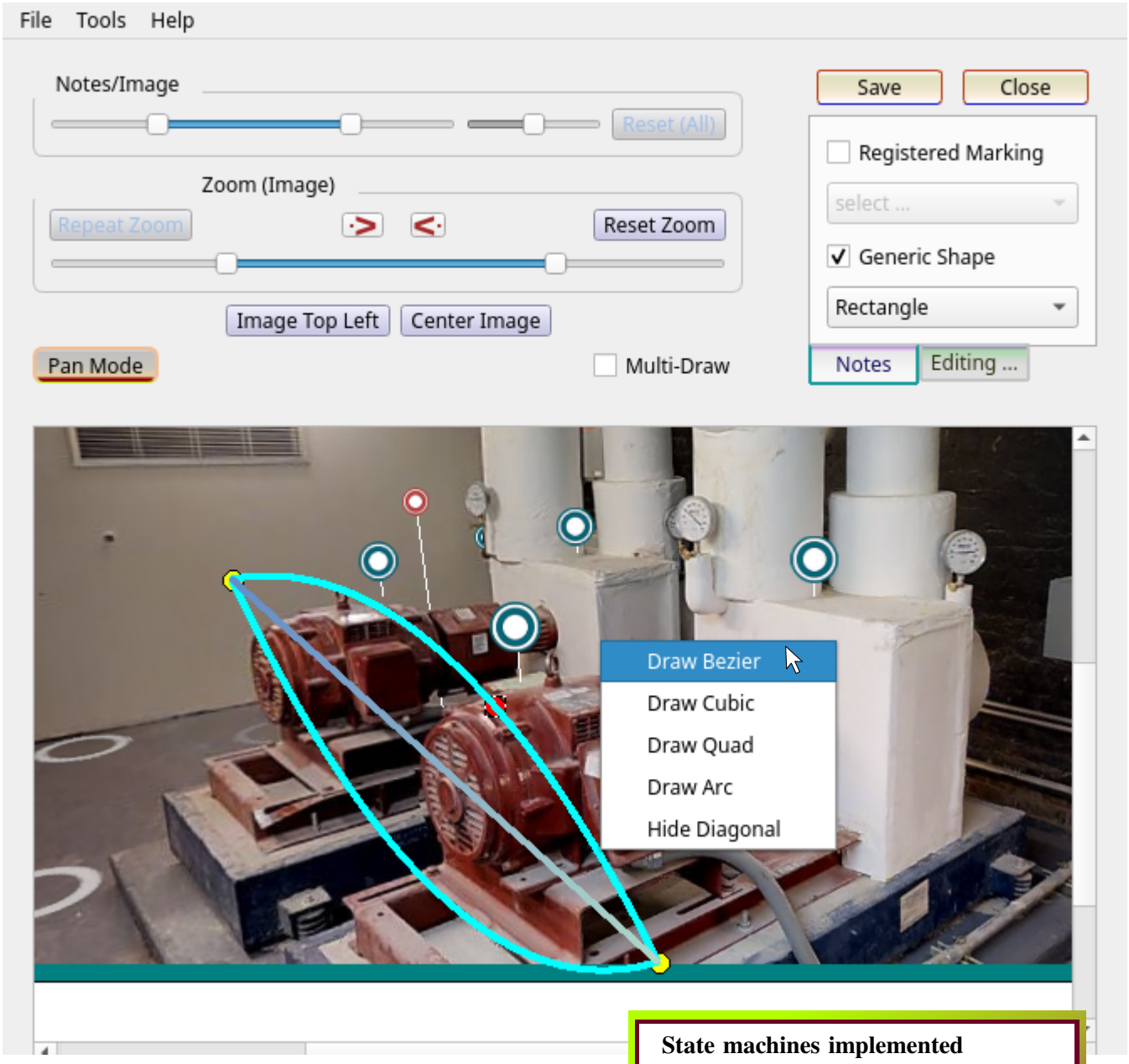
With an integrated parsing and serialization/deserialization system (orthogonal to each application's native data formats) **ConceptsDB*i*** application networks have a common framework for passing data between multiple host/target applications.

The VM-based mechanism for exposing C++ functions to query engines (referenced on the previous two slides) can also be used to introduce numerical formulae for curve shapes and attributes. **ConceptsDB*i*** supports polygon/elliptic annotations (whose shape can be defined with a small set of points and/or magnitudes) plus more complex curves.



The distinction between polygons/ellipses and other curves lies not only in their shape-descriptions, but also in how their visual display integrates with underlying rendering engines — for instance, views based on Qt Graphics Scenes can expose simple shapes as Scene Objects, but must redefine graphics paint events for more complex curves (especially with animations and/or color changes tracked to spline parameters).

VM-backed workflows are also convenient when marshaling engineering data (such as quaternions and bezier/b-spline curves/surfaces) between different formats. State machines allow annotations to be consolidated into a common format; in many cases the intended annotation format can be inferred by the arity of control-point/length vectors.

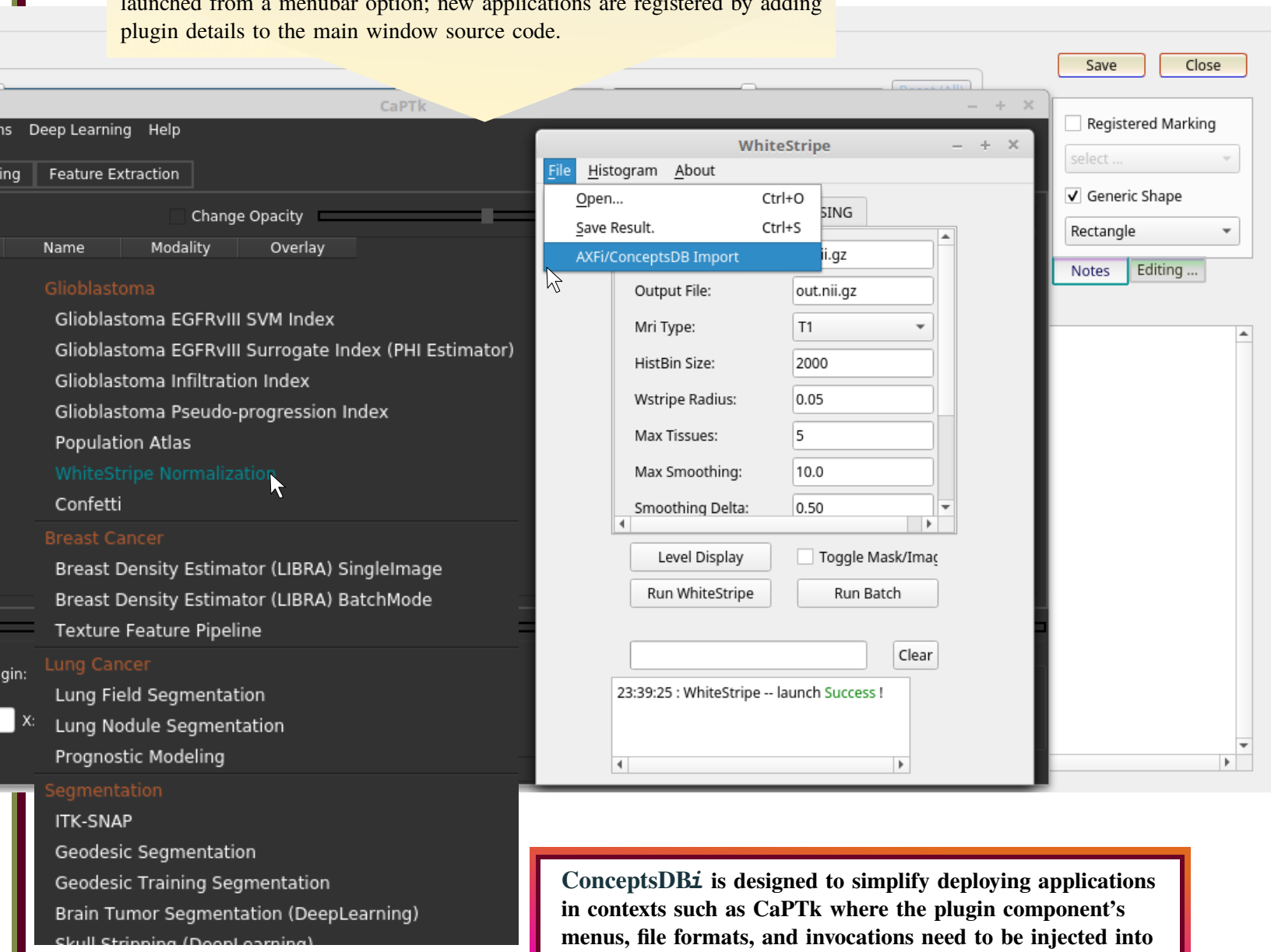


State machines implemented according to these criteria would allow annotation processing to be decomposed into kernel operations suitable for a Virtual Machine, and hence for integration with query-processing frameworks.

The ConceptsDB*i* Virtual-Machine system is also useful when embedding ConceptsDB*i* database support in software environments which rely on semi-autonomous image-processing applications to provide special-purpose functionality.

Use case 13:
CaPTk Integration

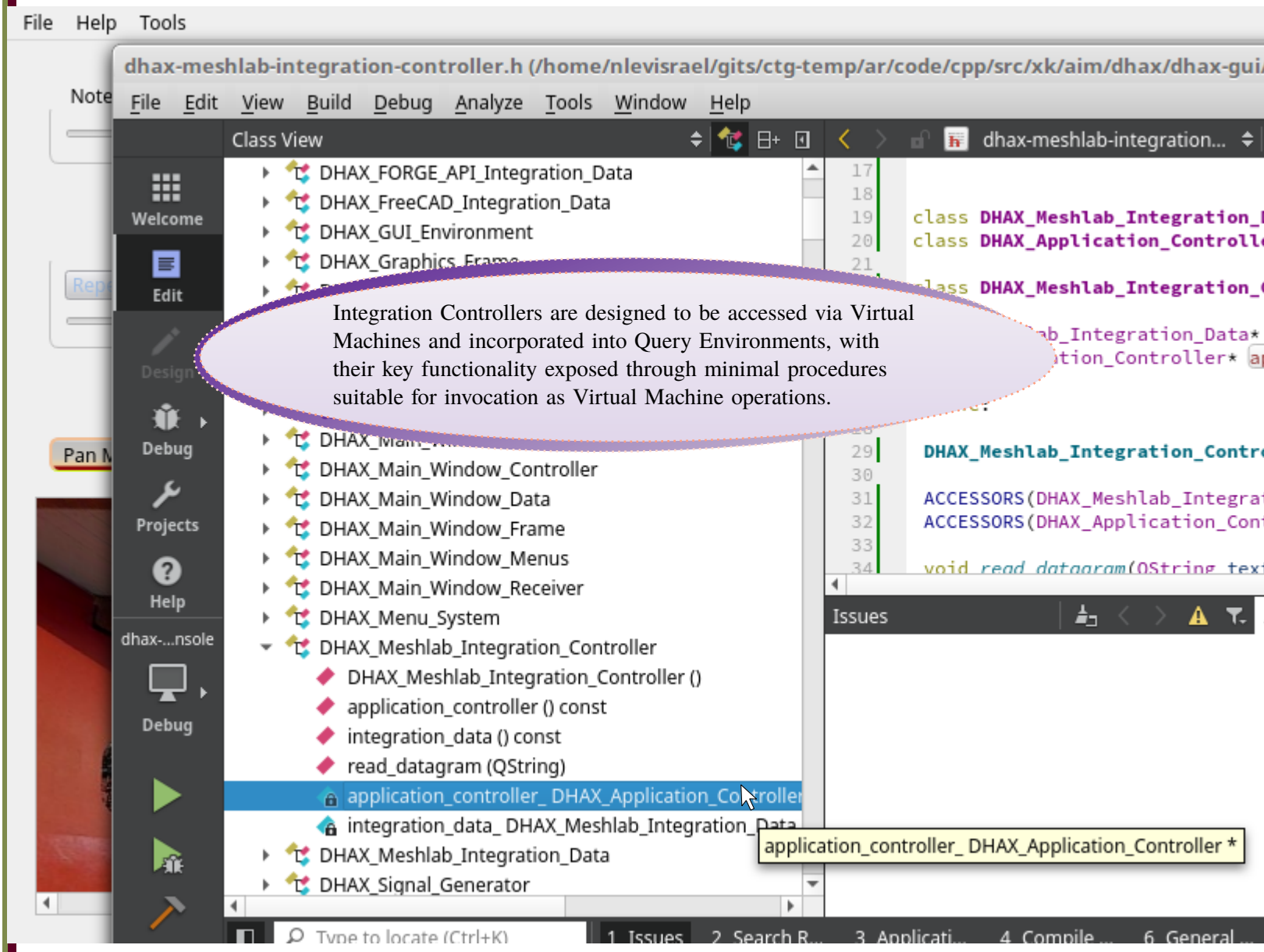
In the CaPTk bioimaging platform, for example, plugin applications can be launched from a menubar option; new applications are registered by adding plugin details to the main window source code.



ConceptsDB*i* is designed to simplify deploying applications in contexts such as CaPTk where the plugin component's menus, file formats, and invocations need to be injected into host environments alongside many other components.

To facilitate the integration with external applications (such as MeshLab or FreeCAD) and the embedding in host applications (such as CaPTk), **ConceptsDBi** introduces "Integration Controllers," which are C++ classes that encapsulate functionality needed to interoperate with the specific external application that they target.

Use case 14:
Integration Controllers



Integration Controllers are designed to be accessed via Virtual Machines and incorporated into Query Environments, with their key functionality exposed through minimal procedures suitable for invocation as Virtual Machine operations.

Re-implementing individual operations allows developers to fine-tune integration capabilities to support the needs of specific projects while reusing all other code in the relevant Integration Controller.

TAKEAWAYS

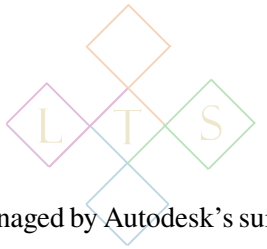
An Application Network implemented via **ConceptsDBi** may involve components such as the following:

QtWebEngine (for GIS mapping and Panoramic Photography): This component would serve as a container for displaying WebGL-based content such as Street Views and Virtual Tours of buildings or outdoor environments. Via **ConceptsDBi**, images could be extracted from these views/tours and saved in an image database, perhaps with additional descriptive tags and comments. A **ConceptsDBi** database could then store and track these annotated images along with contextual data related to geospatial locations, hyperlinks, and textual content (such as Matterport Tags) relevant to how the image was obtained.

MeshLab (3D graphics engine): A common example of software used for viewing engineering, industrial-design, or architectural models in **3D** is the **MeshLab** application. Using **ConceptsDBi**, image databases can acquire **2D** views onto **3D** models rendered through **MeshLab** and save them in a database along with comments and annotations, resulting in tagged descriptive image-sets which complement the original **3D** models. **ConceptsDBi** provides plug-ins/extensions which allow **2D** views to be extracted from software such as **MeshLab**, while also tracking contextual data (for instance, object rotation and camera position) that describe the coordinates through which each visible **2D** graphic is acquired from an ambient **3D** model. The **ConceptsDBi** extension would also be able to obtain contextual data from images in a database and, leveraging this data, recreate the **MeshLab** application state (such as rotation and view settings) through which the image was initially acquired. **ConceptsDBi** could thereby serve as a general-purpose **3D** annotation system.

FreeCAD and OpenSCAD (3D CAD engines): Popular applications for manipulating **3D** graphics in a **CAD** setting include **FreeCAD** and **OpenSCAD**. As with **MeshLab**, **ConceptsDBi** would obtain **2D** views onto **CAD** models and export them to an application for tagging, descriptions, annotations, and finally storage in an image/multimedia database. Similar to **MeshLab**, **ConceptsDBi** would also store contextual information about the **2D** view in its mathematical relationship to the relevant **3D** model (object rotation and camera position); however, **FreeCAD/OpenSCAD** evince further **CAD**-related contextual data, such as object-construction information, which may also be stored in the image database.

Industry Foundation Classes (IFC): Software for managing Industry Foundation data (used in many industrial and architectural contexts), such as **IfcOpenShell**, provides utilities for converting **3D** models between different formats and for linking **3D** objects with engineering data, such as Building Information Management (**BIM**) packages. A **ConceptsDBi** network could include **IfcOpenShell** so as to read information from **IFC** files to be stored in an image database, as well as extract **3D** models from **IFC** files to be viewed in applications such as **MeshLab** or **IfcOpenShell**. **ConceptsDBi** can seamlessly interconnect these applications so that a model obtained from **IfcOpenShell** can be automatically sent to and viewed within (for instance) **MeshLab**.



Forge API: Forge is the API used by Autodesk for team sharing/editing of CAD objects managed by Autodesk's suite of applications. In the context of **ConceptsDBi**, access to the Forge API can be useful for obtaining **CAD** representations which are the basis of images and metadata stored in a database instance. Forge images can thereby be loaded into **ConceptsDBi** applications in a manner resembling the import of **3D** views from **CAD**, although this form of data-integration calls for a significantly different coding paradigm because Forge API access requires a carefully designed Secure Sockets protocol and networking workflow (see slides 14-18 above). In addition, the Forge API provides cloud-hosting services which **ConceptsDBi** can employ for backup and multi-user sharing of database objects.

NCN (Cloud-Native Image Hosting): **NCN**, or "Native Cloud-Native," is LTS's cloud-hosting software, which can be used to provide cloud storage and multi-user sharing for images. Cloud storage allows all personnel involved with a project to be able to view and, as appropriate, make notes or comments about shared images. A distinguishing feature of **NCN** is that, unlike conventional image-hosting services, **NCN** cloud servers are build around the same code libraries and programming environments that are utilized for desktop software, so that applications such as **MeshLab** and **FreeCAD** can seamlessly interoperate with an associated cloud back-end. **NCN** services may directly embed parsers for common industry formats such as **IFC** or **AIM** (Annotation and Image Markup), which means that data can be shared between applications and cloud end-points without translating between different formats.

AXFi (Annotation Exchange Format for Images): The **AXFi** editor component, an internal part of **ConceptsDBi**, is a central window or application for describing and saving images. **AXFi** can be used as a standalone application or embedded as one window into other applications. Images acquired from other software components on a **ConceptsDBi** Application Network would typically be exported to the **AXFi** editor as a central processing location where images could be examined and annotated before being stored in an image database. In addition to graphic annotations (overlays such as polygons, ellipses, and arrows) and free-form textual comments, the **AXFi** editor can interoperate with a domain-specific database and/or utilize Controlled Vocabularies to notate images via machine-readable terms and data insertions. For example, the **AXFi** editor might be extended with dialog boxes allowing users to select terms from a list of recognized keywords in a Controlled Vocabulary. The **AXFi** editor may also describe images with text content that would include results of database queries (for example, strings such as part numbers, **GIS** locations, Object-Oriented data fields, and so forth).

OpenCV (for image processing): Images viewed in the **AXFi** editor and/or persisted in a **ConceptsDBi** image database can be sent to image-processing pipelines, implemented via image-analysis frameworks such as **OpenCV** (Open Computer Vision). These pipelines would typically manipulate the underlying images (for instance, applying blurring or grayscale effects) and then perform analytic routines, such as contour-extraction, yielding statistical data that could be used to process the original image via statistical/machine-learning approaches, such as measuring similarity across multiple images in a database, or extracting Regions of Interest for subsequent region-based tagging and searching.